AI-Based Solar Panel Inspection and Decision Support System

Emerging Trends in Artificial Intelligence Project Report

Will O'Driscoll - Brady Hinks

April 13, 2026

1 Overview

As solar systems scale up, manual inspection becomes harder to justify. It takes time, costs money, and results can vary depending on who is doing the inspection. In a real setting, it is not enough to just label an image correctly. The result needs to be understandable, easy to review, and useful for making decisions.

The goal of this project was to build an AI-assisted system that inspects solar panel images and supports decision making around maintenance. The problem we are solving is the effort and inconsistency involved in manual inspection of panel conditions like dust, bird droppings, snow, physical damage, and electrical damage.

Instead of treating this as just an image classification task, we approached it as a full system. That meant handling the dataset, training the model, running inference, generating explanations, structuring outputs, and building a dashboard someone could actually use. Our specific role covered the full pipeline end-to-end, including dataset preparation, model training with ResNet50, the inference and Grad-CAM backend, the rule-based risk scoring layer, and the Streamlit dashboard that ties it all together.

2 System Design

The system is split into two main parts: a backend that does all the model work, and a frontend dashboard that handles review.

2.1 Main components

- **Backend pipeline** (`backend/app/pipeline`). This loads the model, preprocesses images, runs inference, generates Grad-CAM, computes risk, and writes results.
- **Model artifacts** (`models/checkpoints`). Holds the trained checkpoint (`TestModel.pth`) and the class order file (`classes.txt`).
- **Dashboard** (`dashboard/app.py` and `dashboard/ui`). The Streamlit interface for uploading images and reviewing outputs.
- **Results store** (`dashboard/results`). Structured output files the frontend reads back.

2.2 Inputs and outputs

Input: a single solar panel image uploaded through the dashboard.

Output: a structured result containing the predicted class, confidence, fault probability, risk level (HIGH, MEDIUM, LOW, or REVIEW), and a Grad-CAM heatmap overlay.

2.3 How the parts connect

The frontend is kept deliberately thin. When a user uploads an image, the dashboard calls the backend pipeline, which runs inference and returns a structured result plus visualization. The dashboard then loads that result from the results store and renders it across the overview, drilldown, and visualization tabs. All model logic stays in the backend, and the frontend is purely for reviewing results.

2.4 Dataset

The dataset includes six classes:

- Bird-drop
- Clean
- Dusty
- Electrical-damage
- Physical-Damage

- Snow-Covered

Table 1: Dataset distribution

Class	Training	Validation
Bird-drop	177	104
Clean	169	102
Dusty	162	97
Electrical-damage	135	77
Physical-Damage	132	78
Snow-Covered	154	92

The dataset is not perfectly balanced, but it is not extreme either. Still, even moderate imbalance can affect how the model behaves, so it is something we kept in mind.

3 Method / Approach

3.1 Modeling

We used a pretrained ResNet50 model and replaced the final layer for our six classes. The backbone is frozen and only the final layer is trained. This is a standard transfer learning setup and it made sense here because the dataset is not huge, and ResNet50 already knows how to extract strong general-purpose visual features.

Training setup:

- image resizing to 224×224,
- data augmentation,
- normalization,
- cross-entropy loss,
- Adam optimizer.

We also save the class order alongside the checkpoint, which is critical when moving the model into an application. Without it, the predicted index has no reliable meaning.

3.2 Preprocessing and explainability

Right now, preprocessing is basic. We resize and normalize images, but we are not doing strong background removal yet. That is something that needs improvement.

Explainability is in better shape. We generate Grad-CAM heatmaps for each prediction so the user can see which parts of the panel the model is focusing on. If Grad-CAM fails for some reason, we flag the result for review instead of pretending everything worked.

3.3 Decision logic

Instead of just showing raw predictions, we convert outputs into something more useful:

- predicted class,
- confidence,
- fault probability,
- risk level.

Risk rules:

- HIGH: probability > 0.78
- MEDIUM: 0.45 to 0.78
- LOW: ≤ 0.45
- REVIEW: low confidence or explainability failure

This is a simple rule-based layer on top of the model. It is not perfect, but it is clear, easy to adjust, and gives the user something actionable instead of a raw probability.

4 Implementation

4.1 Tools and libraries

- **PyTorch and torchvision** for model definition, loading pretrained ResNet50, and training.

- **Streamlit** for the dashboard frontend.
- **Pillow and OpenCV** for image loading and preprocessing.
- **NumPy and pandas** for numeric handling and result tables.
- **Matplotlib** for Grad-CAM heatmap rendering.

4.2 Important parts of the implementation

- `backend/app/pipeline/model.py` loads the ResNet50 checkpoint and handles fallback for different save formats (full model vs state dict).
- `backend/app/pipeline/inference.py` runs a prediction and returns class, confidence, and fault probability.
- `backend/app/pipeline/gradcam.py` generates the heatmap overlay.
- `backend/app/pipeline/artifacts.py` writes structured results so the dashboard can read them back.
- `dashboard/app.py` and `dashboard/ui/` handle Streamlit layout, components, and results IO.

4.3 Challenges during development

- **Checkpoint format inconsistencies.** Models saved in different ways (full object vs state dict) broke loading. We added fallback handling so the backend tries both.
- **Class order drift.** If the training-time class order is lost, predictions become meaningless. We now save and load `classes.txt` next to the checkpoint.
- **Grad-CAM failures.** Grad-CAM can fail quietly on certain inputs. Instead of hiding it, we flag the result as REVIEW so the user knows something is off.
- **Keeping frontend and backend separate.** It was tempting to put model logic into the Streamlit app, but we kept it clean and forced everything through the pipeline.

5 Results / Output

The system produces structured per-image results that the dashboard renders across a few views.

5.1 What the dashboard shows

- **Upload view.** User drops in an image.
- **Overview.** Predicted class, confidence, fault probability, and risk level.
- **Drilldown.** Original image, Grad-CAM heatmap, and overlay.
- **Run history.** Previous inspections stored in the results folder.

5.2 Example output fields

For each inspection, the backend writes a structured record such as:

- predicted class: `Dusty`
- confidence: `0.87`
- fault probability: `0.82`
- risk level: `HIGH`
- Grad-CAM: generated successfully

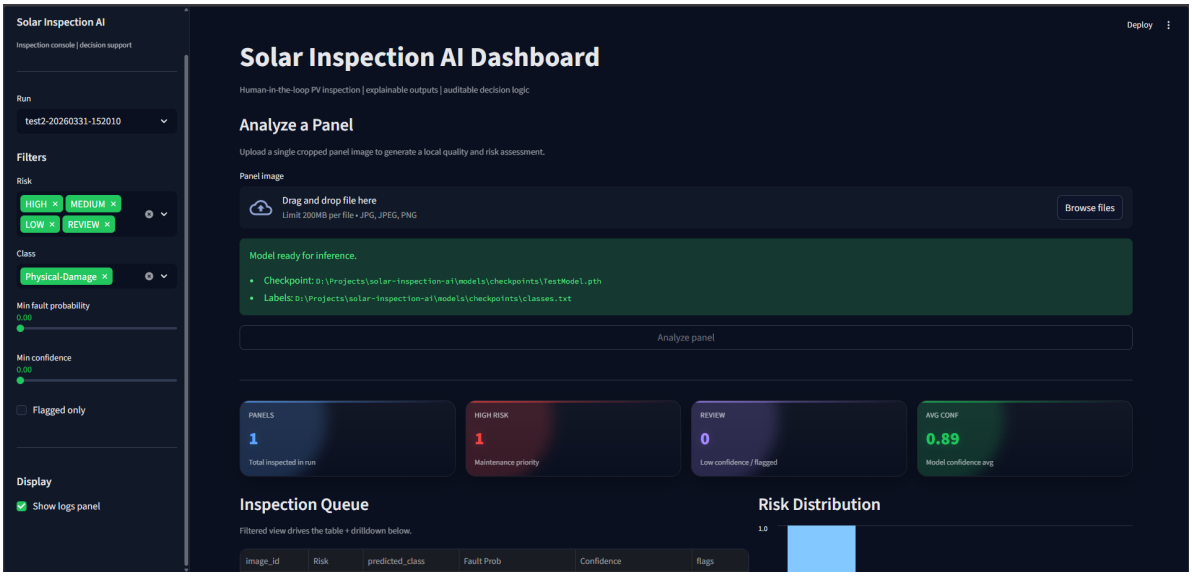

Figure 1: Main dashboard showing the upload panel, model status, run-level metrics (panels inspected, high risk, review, average confidence), and risk filters.

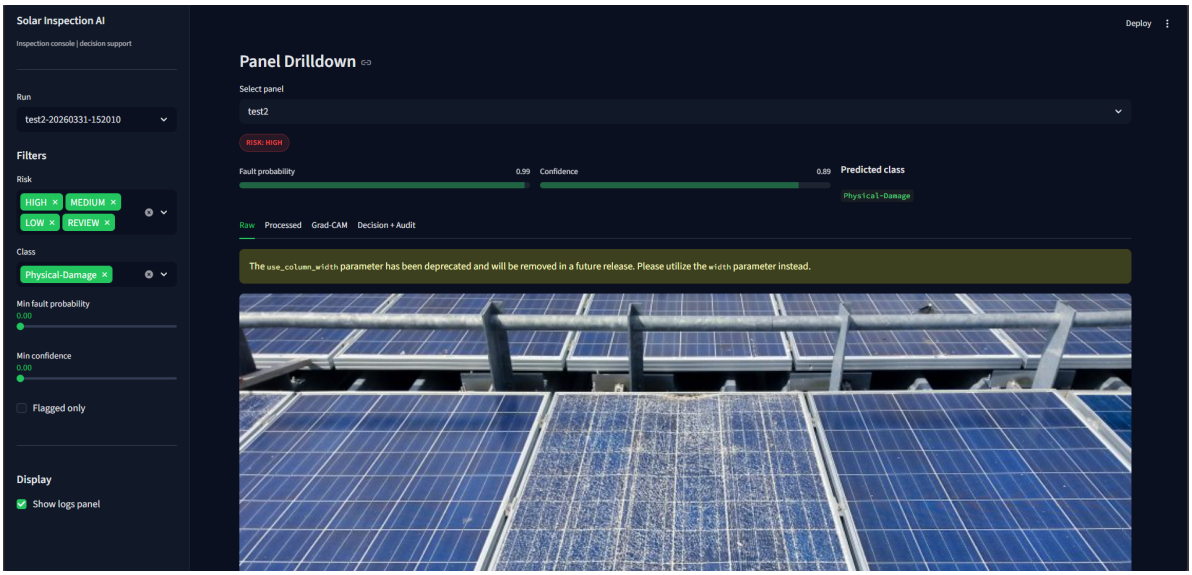

Figure 2: Panel drilldown with fault probability, confidence, and predicted class for the selected panel, along with tabs for the original image, processed input, Grad-CAM, and health.

A trained checkpoint (`TestModel.pth`) is committed to the repo and loads cleanly through the backend, so the pipeline runs end-to-end on real inputs. What is still pending is a full evaluation pass on the validation split, which would give us per-class accuracy, a confusion matrix, and proper reporting. Until that is done, we can show the system working, but we cannot make strong claims about how well the model performs across classes.

6 Challenges and Improvements

6.1 Difficulties faced

- Handling inconsistent checkpoint formats without breaking the pipeline.
- Keeping class order aligned between training and inference.
- Dealing with Grad-CAM failures gracefully.
- Moderate class imbalance in the dataset.
- Basic preprocessing that does not isolate the panel from the background.

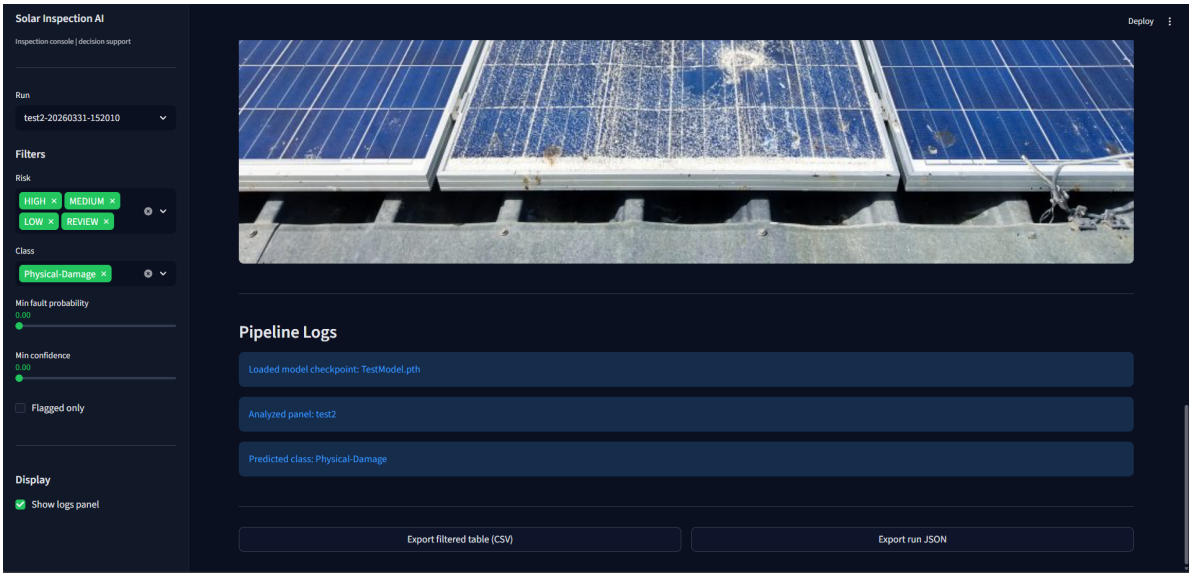

Figure 3: Pipeline logs and export view. Checkpoint load, image read, and predicted class are written out, with CSV and JSON export for the run.

6.2 Possible future improvements

- **Better preprocessing.** Background removal or panel segmentation would reduce bias from scenery and improve reliability.
- **Full evaluation.** Run the committed checkpoint against the validation set and report per-class accuracy and a confusion matrix.
- **Unfreeze more layers.** Once evaluation is in, fine-tuning deeper layers of ResNet50 could help on the weaker classes.
- **Multi-image and batch inspection.** Right now the system is focused on single-image inspection. Batch runs would make it more realistic for site-scale use.
- **Tuning the risk thresholds.** The current thresholds (0.45, 0.78) are a reasonable starting point, but they should be calibrated once real evaluation metrics exist.

7 Conclusion

We built a working AI-assisted solar panel inspection system that goes beyond just model training. It combines a ResNet50 classifier, Grad-CAM explainability, rule-based risk scoring, and a Streamlit dashboard into one end-to-end workflow. A trained checkpoint is committed and the full pipeline runs on real inputs.

The main outcome is a clear path from a trained model to something usable and understandable by a non-ML reviewer. There are still gaps, especially around formal validation metrics and preprocessing, but the system shows how classification, explanation, and decision support can be tied together in a single inspection tool.